

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
«Процедуры, функции, триггеры в PostgreSQL»

По дисциплине «Проектирование и реализация баз данных»

Вариант 11

Обучающийся: Востров Илья Анатольевич

Факультет: ФПИИ

Группа: К3239

Направление подготовки: 09.03.03 Прикладная информатика

Образовательная программа: Мобильные и сетевые технологии

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
Цель работы	3
Практическое задание	3
Выполнение	4
Создание хранимых процедур и функций	4
Процедура 1. Повышение цены деталей для автомобиля «Ford» на 10%	4
Процедура 2. Повышение разряда мастеров, отремонтировавших больше 3 авто.	5
Функция 3. Расчет количества отремонтированных авто каждым механиком за квартал.	7
Создание 7 оригинальных триггеров	8
Триггер 1. Логирование изменения статуса договора.	8
Триггер 2. Запрет на удаление активных договоров.	10
Триггер 3. Автоматический расчет общей стоимости услуги	11
Триггер 4. Списание деталей со склада.	13
Триггер 5. Авто-форматирование госномера машины.	14
Триггер 6. Аудит изменения разряда механиков.	15
Триггер 7. Защита от отрицательной стоимости работ.	16
ВЫВОДЫ	17

Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2).
2. Создать 7 оригинальных триггеров для индивидуальной БД (Вариант 2.2).

Все задания выполнялись в консольном клиенте psql.

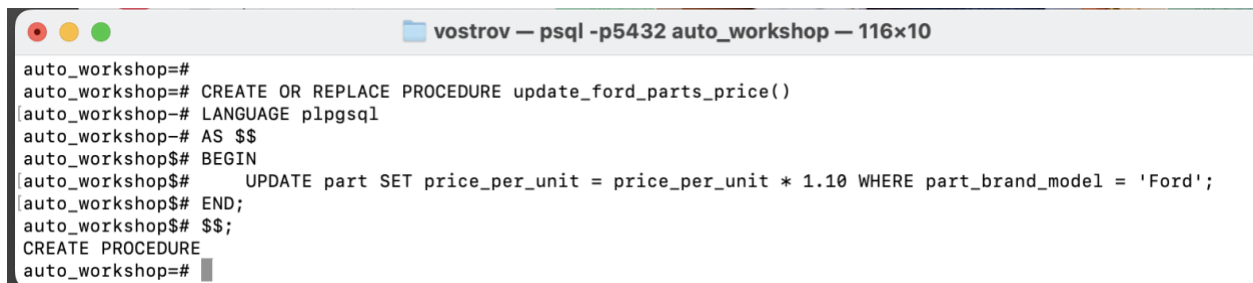
Выполнение

Создание хранимых процедур и функций

Процедура 1. Повышение цены деталей для автомобиля «Ford» на 10%

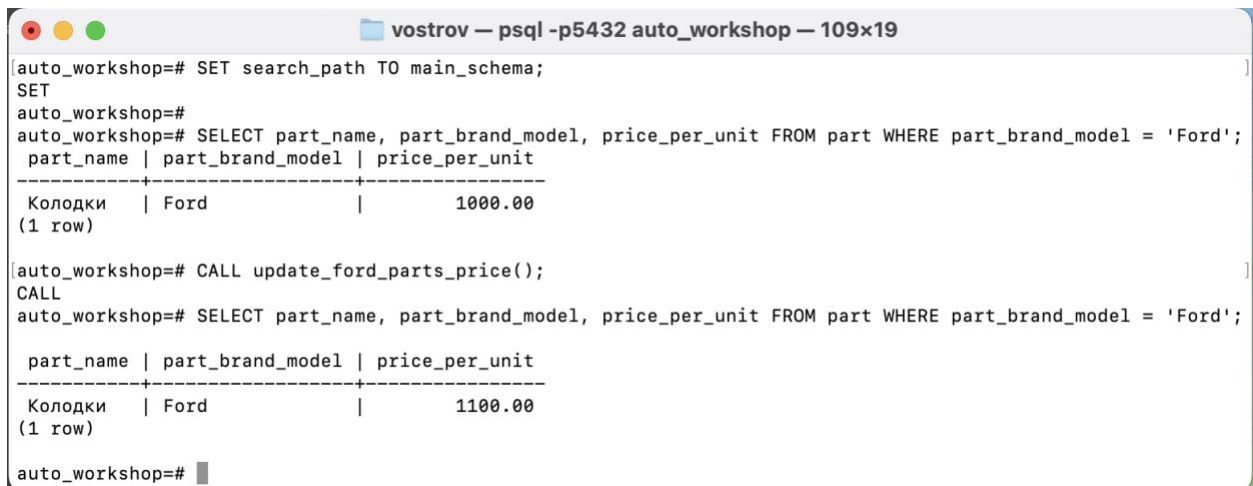
Цель: Массово обновить цены в справочнике деталей на основе условия

Что я сделал: Написал процедуру `update_ford_parts_price`, инкапсулирующую команду `UPDATE`.



```
vostrov — psql -p5432 auto_workshop — 116x10
auto_workshop=#
auto_workshop=# CREATE OR REPLACE PROCEDURE update_ford_parts_price()
[auto_workshop=# LANGUAGE plpgsql
auto_workshop=# AS $$
auto_workshop$# BEGIN
[auto_workshop$# UPDATE part SET price_per_unit = price_per_unit * 1.10 WHERE part_brand_model = 'Ford';
[auto_workshop$# END;
auto_workshop$# $$;
CREATE PROCEDURE
auto_workshop=#
```

Рисунок 1 — Процедура



```
vostrov — psql -p5432 auto_workshop — 109x19
auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# SELECT part_name, part_brand_model, price_per_unit FROM part WHERE part_brand_model = 'Ford';
part_name | part_brand_model | price_per_unit
-----+-----+-----
Колодки   | Ford             | 1000.00
(1 row)

auto_workshop=# CALL update_ford_parts_price();
CALL
auto_workshop=# SELECT part_name, part_brand_model, price_per_unit FROM part WHERE part_brand_model = 'Ford';

part_name | part_brand_model | price_per_unit
-----+-----+-----
Колодки   | Ford             | 1100.00
(1 row)

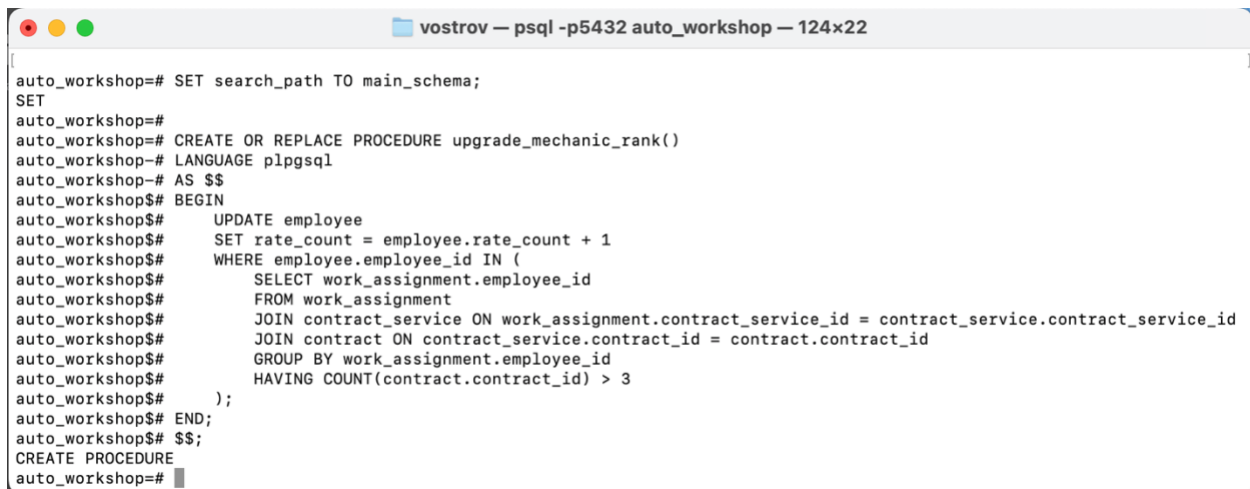
auto_workshop=#
```

Рисунок 2 — Результат

Процедура 2. Повышение разряда мастеров, отремонтировавших больше 3 авто.

Цель: Изменить рейтинг сотрудника на основе аналитики из связанных таблиц.

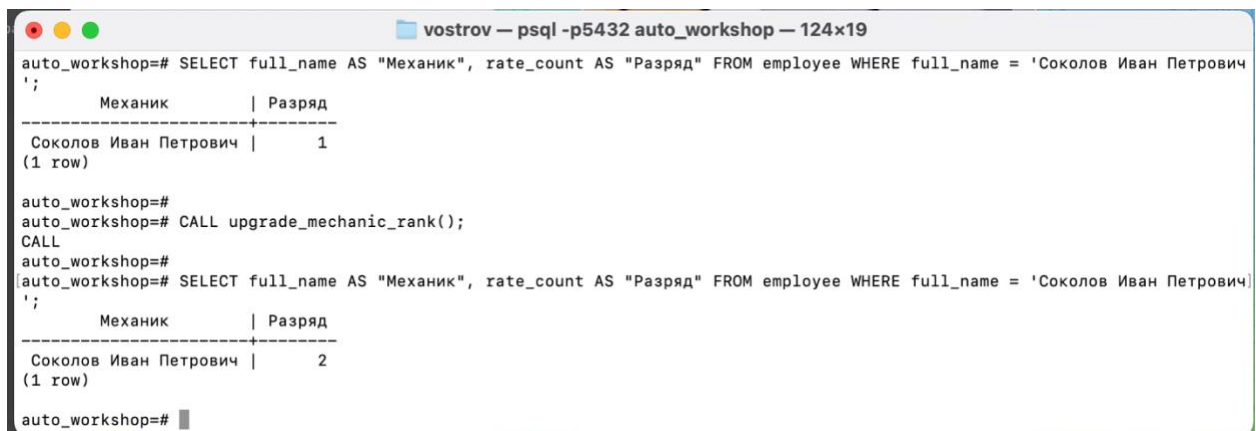
Что я сделал: Написал процедуру `upgrade_mechanic_rank`. Для поиска целевых сотрудников использовал вложенный подзапрос с агрегацией `HAVING COUNT > 3`.



```
vostrov — psql -p5432 auto_workshop — 124x22

[
auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# CREATE OR REPLACE PROCEDURE upgrade_mechanic_rank()
auto_workshop=# LANGUAGE plpgsql
auto_workshop=# AS $$
auto_workshop$# BEGIN
auto_workshop$#     UPDATE employee
auto_workshop$#     SET rate_count = employee.rate_count + 1
auto_workshop$#     WHERE employee.employee_id IN (
auto_workshop$#         SELECT work_assignment.employee_id
auto_workshop$#         FROM work_assignment
auto_workshop$#         JOIN contract_service ON work_assignment.contract_service_id = contract_service.contract_service_id
auto_workshop$#         JOIN contract ON contract_service.contract_id = contract.contract_id
auto_workshop$#         GROUP BY work_assignment.employee_id
auto_workshop$#         HAVING COUNT(contract.contract_id) > 3
auto_workshop$#     );
auto_workshop$# END;
auto_workshop$# $$;
CREATE PROCEDURE
auto_workshop=# ]
```

Рисунок 3 — Процедура



```
vostrov — psql -p5432 auto_workshop — 124x19

auto_workshop=# SELECT full_name AS "Механик", rate_count AS "Разряд" FROM employee WHERE full_name = 'Соколов Иван Петрович';
   Механик   | Разряд 
-----+-----
Соколов Иван Петрович |      1
(1 row)

auto_workshop=#
auto_workshop=# CALL upgrade_mechanic_rank();
CALL
auto_workshop=#
auto_workshop=# SELECT full_name AS "Механик", rate_count AS "Разряд" FROM employee WHERE full_name = 'Соколов Иван Петрович';
   Механик   | Разряд 
-----+-----
Соколов Иван Петрович |      2
(1 row)

auto_workshop=# ]
```

Рисунок 4 — Результат

```
vostrov — psql -p5432 auto_workshop — 124x19
(1 row)

auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# SELECT
auto_workshop=#     employee.full_name AS "Механик",
auto_workshop=#     employee.rate_count AS "Текущий разряд",
auto_workshop=#     COUNT(work_assignment.assignment_id) AS "Отремонтировано авто"
auto_workshop=# FROM employee
auto_workshop=# LEFT JOIN work_assignment ON employee.employee_id = work_assignment.employee_id
auto_workshop=# WHERE employee.full_name = 'Соколов Иван Петрович'
auto_workshop=# GROUP BY employee.employee_id, employee.full_name, employee.rate_count;
]
    Механик      | Текущий разряд | Отремонтировано авто
-----+-----+-----
Соколов Иван Петрович |          2    |          4
(1 row)

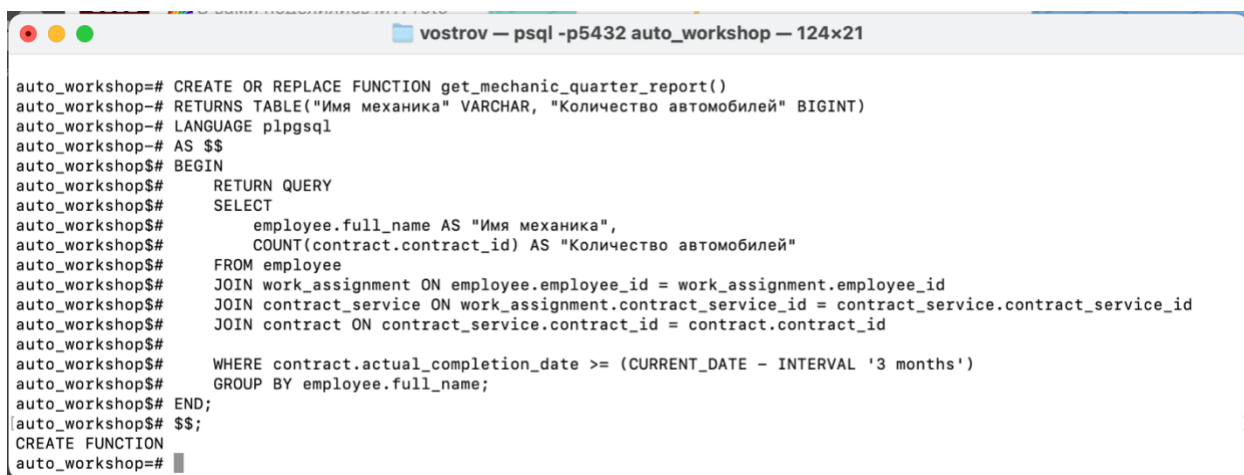
auto_workshop=#
```

Рисунок 5 — Доп результат

Функция 3. Расчет количества отремонтированных авто каждым механиком за квартал.

Цель: Сформировать сводную таблицу на основе временного периода.

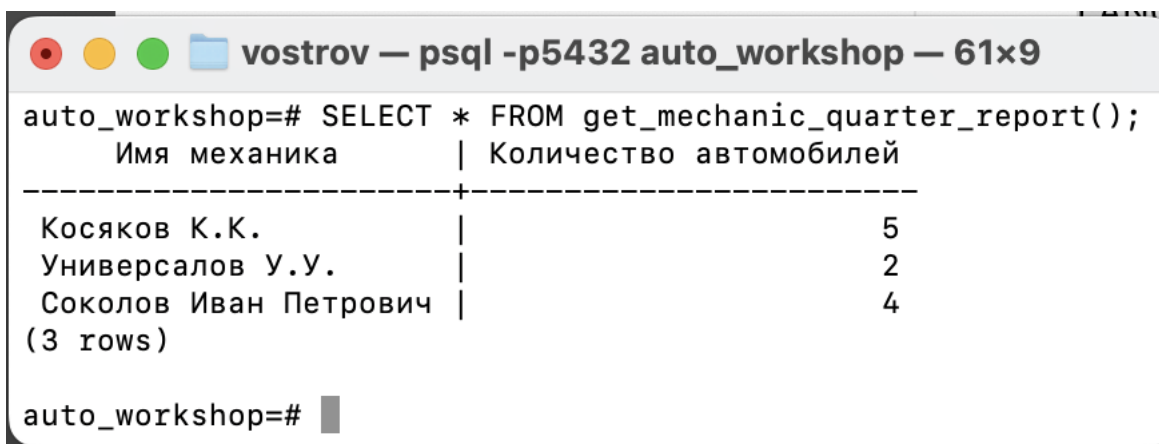
Что я сделал: Создал функцию `get_mechanic_quarter_report()`, возвращающая тип `TABLE`. В теле функции используется динамический интервал `CURRENT_DATE - INTERVAL '3 months'`.



```
vostrov — psql -p5432 auto_workshop — 124x21

auto_workshop=# CREATE OR REPLACE FUNCTION get_mechanic_quarter_report()
auto_workshop=# RETURNS TABLE("Имя механика" VARCHAR, "Количество автомобилей" BIGINT)
auto_workshop=# LANGUAGE plpgsql
auto_workshop=# AS $$
auto_workshop$# BEGIN
auto_workshop$#     RETURN QUERY
auto_workshop$#     SELECT
auto_workshop$#         employee.full_name AS "Имя механика",
auto_workshop$#         COUNT(contract.contract_id) AS "Количество автомобилей"
auto_workshop$#     FROM employee
auto_workshop$#     JOIN work_assignment ON employee.employee_id = work_assignment.employee_id
auto_workshop$#     JOIN contract_service ON work_assignment.contract_service_id = contract_service.contract_service_id
auto_workshop$#     JOIN contract ON contract_service.contract_id = contract.contract_id
auto_workshop$#     WHERE contract.actual_completion_date >= (CURRENT_DATE - INTERVAL '3 months')
auto_workshop$#     GROUP BY employee.full_name;
auto_workshop$# END;
auto_workshop$# $$;
auto_workshop=# CREATE FUNCTION
auto_workshop=#
```

Рисунок 6 — Функция



```
vostrov — psql -p5432 auto_workshop — 61x9

auto_workshop=# SELECT * FROM get_mechanic_quarter_report();
  Имя механика  | Количество автомобилей 
-----+-----
  Косяков К.К.  |                      5
  Универсалов У.У. |                      2
  Соколов Иван Петрович |                      4
(3 rows)

auto_workshop=#
```

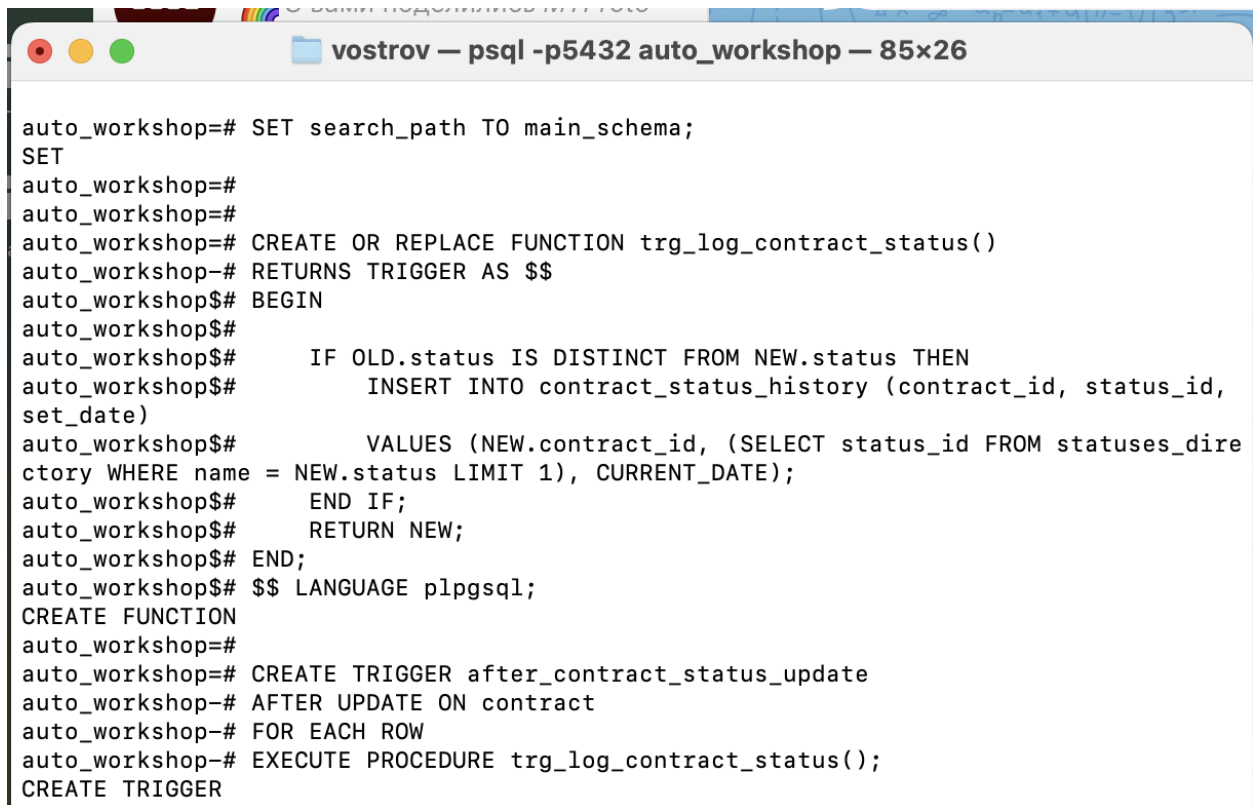
Рисунок 7 — Результат

Создание 7 оригинальных триггеров

Триггер 1. Логирование изменения статуса договора.

Цель: Автоматически сохранять историю смены статусов заказа.

Что я сделал: Триггер AFTER UPDATE. Если старый статус не равен новому, функция добавляет запись в таблицу contract_status_history.

A screenshot of a PostgreSQL terminal window. The title bar shows 'vostrov — psql -p5432 auto_workshop — 85x26'. The terminal contains SQL commands to create a function and a trigger. The function 'trg_log_contract_status' is created with the LANGUAGE plpgsql and returns a trigger. It contains logic to insert a record into 'contract_status_history' when the status of a contract changes. The trigger 'after_contract_status_update' is then created to fire after an UPDATE on the 'contract' table.

```
auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=#
auto_workshop=# CREATE OR REPLACE FUNCTION trg_log_contract_status()
auto_workshop=# RETURNS TRIGGER AS $$
auto_workshop$$ BEGIN
auto_workshop$$
auto_workshop$$     IF OLD.status IS DISTINCT FROM NEW.status THEN
auto_workshop$$         INSERT INTO contract_status_history (contract_id, status_id,
auto_workshop$$ set_date)
auto_workshop$$             VALUES (NEW.contract_id, (SELECT status_id FROM statuses_directory WHERE name = NEW.status LIMIT 1), CURRENT_DATE);
auto_workshop$$     END IF;
auto_workshop$$     RETURN NEW;
auto_workshop$$ END;
auto_workshop$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
auto_workshop=#
auto_workshop=# CREATE TRIGGER after_contract_status_update
auto_workshop=# AFTER UPDATE ON contract
auto_workshop=# FOR EACH ROW
auto_workshop=# EXECUTE PROCEDURE trg_log_contract_status();
CREATE TRIGGER
```

Рисунок 8 — Триггер

```
vostrov — psql -p5432 auto_workshop — 85x17
auto_workshop=# SELECT * FROM contract_status_history;
 id_record | contract_id | status_id | set_date
-----+-----+-----+-----
(0 rows)

auto_workshop=#
auto_workshop=# UPDATE contract SET status = 'Готово' WHERE contract_id = (SELECT min
(contract_id) FROM contract);
UPDATE 1
auto_workshop=#
auto_workshop=# SELECT * FROM contract_status_history;
 id_record | contract_id | status_id | set_date
-----+-----+-----+-----
          1 |          1 |          1 | 2026-04-19
(1 row)

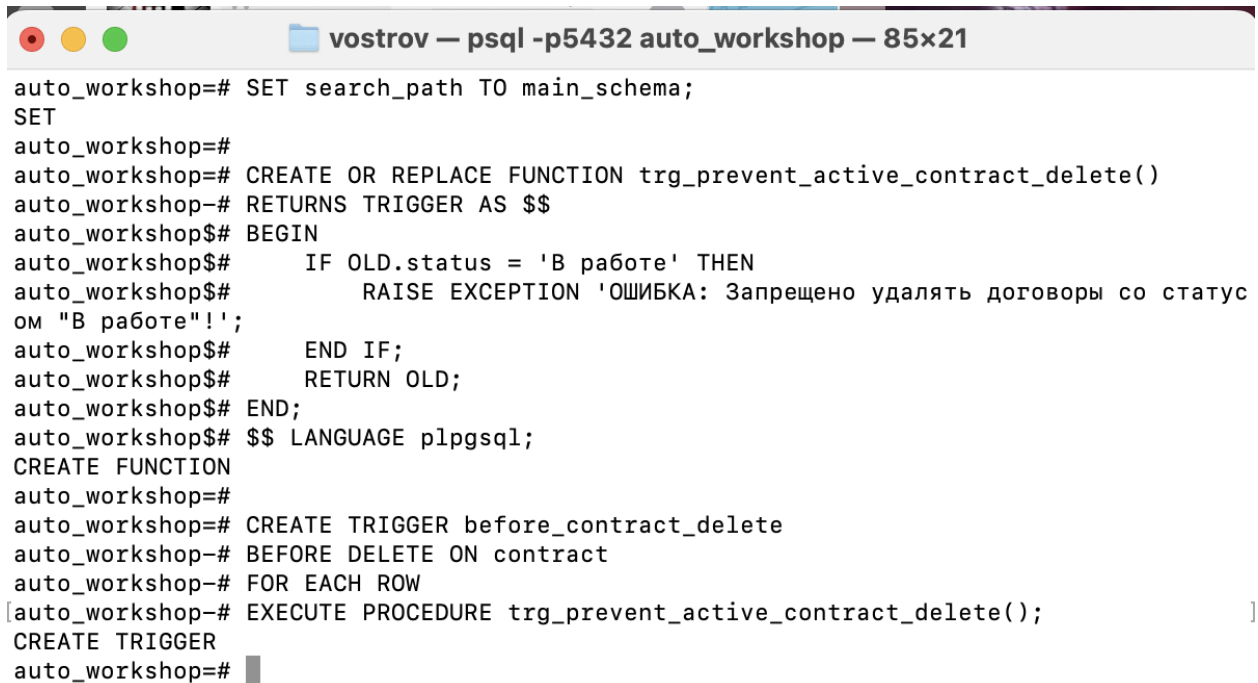
auto_workshop=#
```

Рисунок 9 — Результат

Триггер 2. Запрет на удаление активных договоров.

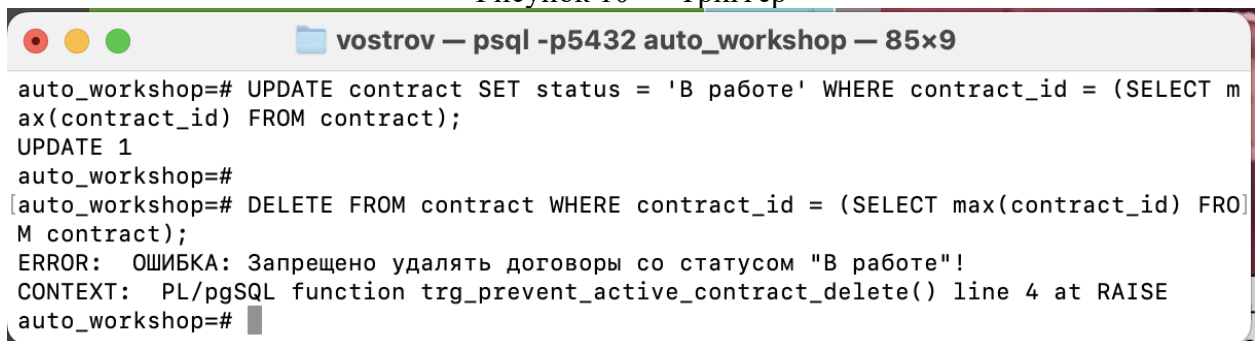
Цель: Защитить БД от случайной потери актуальных заказов.

Что я сделал: Триггер BEFORE DELETE. Функция проверяет OLD.status. Если он равен 'В работе', вызывается RAISE EXCEPTION и удаление прерывается.



```
vostrov — psql -p5432 auto_workshop — 85x21
auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# CREATE OR REPLACE FUNCTION trg_prevent_active_contract_delete()
auto_workshop=# RETURNS TRIGGER AS $$
auto_workshop$$ BEGIN
auto_workshop$$     IF OLD.status = 'В работе' THEN
auto_workshop$$         RAISE EXCEPTION 'ОШИБКА: Запрещено удалять договоры со статус
ом "В работе"!';
auto_workshop$$     END IF;
auto_workshop$$     RETURN OLD;
auto_workshop$$ END;
auto_workshop$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
auto_workshop=#
auto_workshop=# CREATE TRIGGER before_contract_delete
auto_workshop=# BEFORE DELETE ON contract
auto_workshop=# FOR EACH ROW
auto_workshop=# EXECUTE PROCEDURE trg_prevent_active_contract_delete();
CREATE TRIGGER
auto_workshop=#
```

Рисунок 10 — Триггер



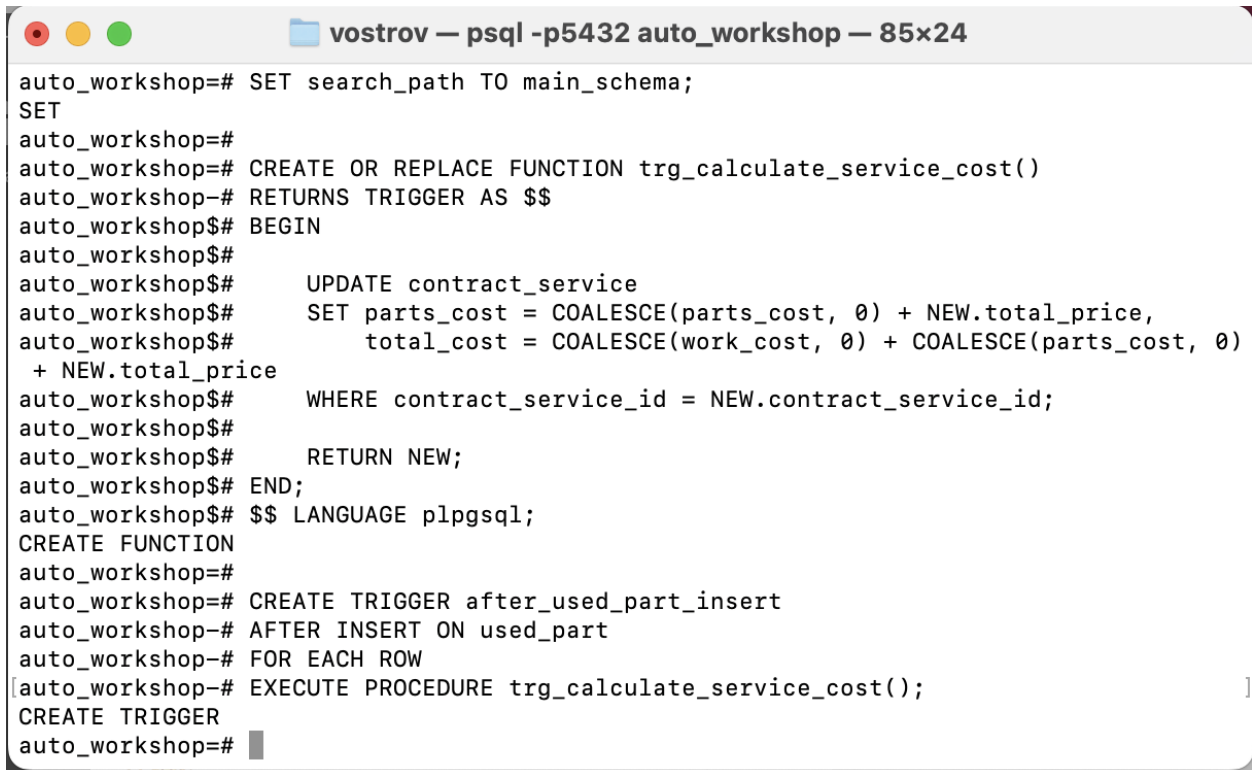
```
vostrov — psql -p5432 auto_workshop — 85x9
auto_workshop=# UPDATE contract SET status = 'В работе' WHERE contract_id = (SELECT m
ax(contract_id) FROM contract);
UPDATE 1
auto_workshop=#
auto_workshop=# DELETE FROM contract WHERE contract_id = (SELECT max(contract_id) FRO
M contract);
ERROR: ОШИБКА: Запрещено удалять договоры со статусом "В работе"!
CONTEXT: PL/pgSQL function trg_prevent_active_contract_delete() line 4 at RAISE
auto_workshop=#
```

Рисунок 11 — Результат

Триггер 3. Автоматический расчет общей стоимости услуги

Цель: Динамически обновлять стоимость услуги при добавлении к ней запчастей.

Что я сделал: Триггер AFTER INSERT на таблицу used_part. Функция берет стоимость добавленной детали и прибавляет её к колонке total_cost связанной услуги.



```
vostrov — psql -p5432 auto_workshop — 85x24
auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# CREATE OR REPLACE FUNCTION trg_calculate_service_cost()
auto_workshop=# RETURNS TRIGGER AS $$
auto_workshop$$ BEGIN
auto_workshop$$
auto_workshop$$      UPDATE contract_service
auto_workshop$$      SET parts_cost = COALESCE(parts_cost, 0) + NEW.total_price,
auto_workshop$$          total_cost = COALESCE(work_cost, 0) + COALESCE(parts_cost, 0)
auto_workshop$$      + NEW.total_price
auto_workshop$$      WHERE contract_service_id = NEW.contract_service_id;
auto_workshop$$      RETURN NEW;
auto_workshop$$ END;
auto_workshop$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
auto_workshop=#
auto_workshop=# CREATE TRIGGER after_used_part_insert
auto_workshop=# AFTER INSERT ON used_part
auto_workshop=# FOR EACH ROW
[auto_workshop=# EXECUTE PROCEDURE trg_calculate_service_cost();
CREATE TRIGGER
auto_workshop=#
```

Рисунок 12 — Триггер

```
vostrov — psql -p5432 auto_workshop — 85x23

auto_workshop=# SELECT contract_service_id, parts_cost, total_cost FROM contract_serv
ice ORDER BY contract_service_id LIMIT 1;
 contract_service_id | parts_cost | total_cost
-----+-----+-----
1 | 500.00 | 10500.00
(1 row)

auto_workshop=#
auto_workshop=# INSERT INTO used_part (contract_service_id, part_id, quantity, total_
price)
auto_workshop=# VALUES ((SELECT min(contract_service_id) FROM contract_service), (SEL
ECT min(part_id) FROM part), 1, 500);
INSERT 0 1
auto_workshop=#
[auto_workshop=# SELECT contract_service_id, parts_cost, total_cost FROM contract_serv]
ice ORDER BY contract_service_id LIMIT 1;
 contract_service_id | parts_cost | total_cost
-----+-----+-----
1 | 1000.00 | 11000.00
(1 row)

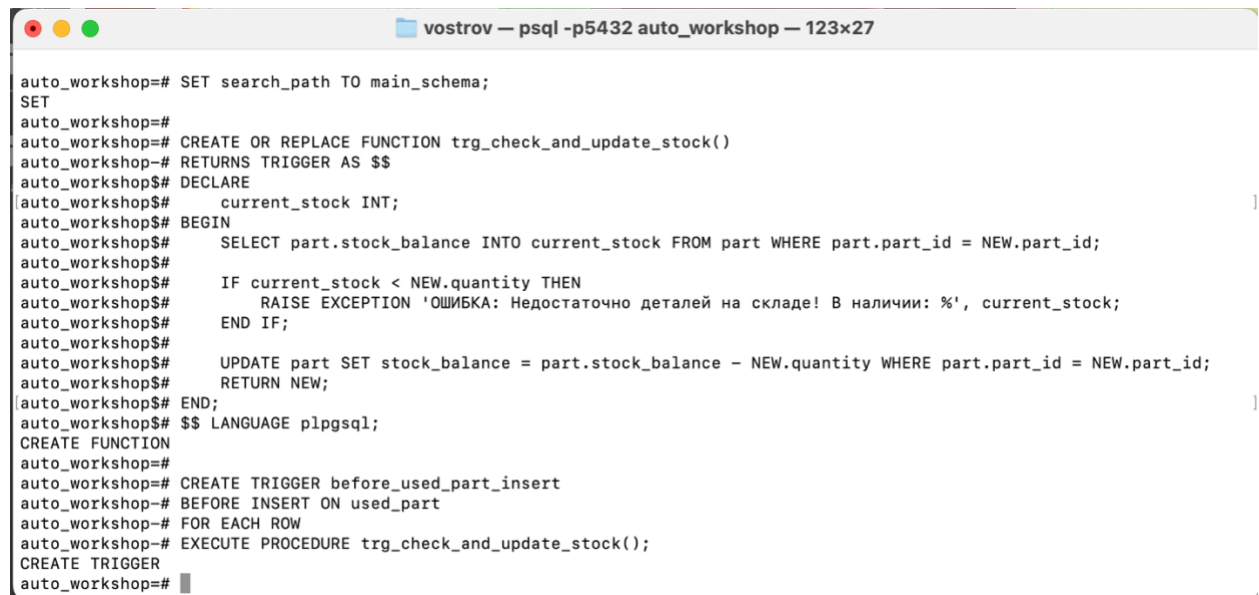
auto_workshop=#
```

Рисунок 13 — Результат

Триггер 4. Списание деталей со склада.

Цель: Обеспечить контроль складских остатков.

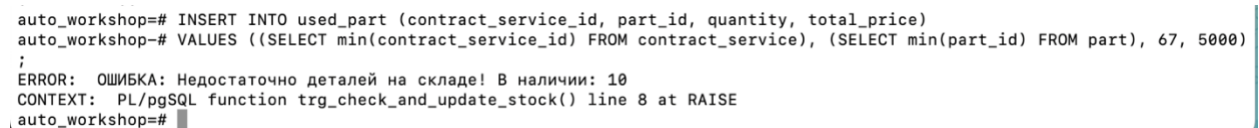
Что я сделал: Триггер BEFORE INSERT. Перед добавлением детали в ремонт функция проверяет stock_balance. Если остаток меньше требуемого — блокирует вставку ошибкой. Иначе — списывает деталь



```
vostrov — psql -p5432 auto_workshop — 123x27

auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# CREATE OR REPLACE FUNCTION trg_check_and_update_stock()
auto_workshop=# RETURNS TRIGGER AS $$
auto_workshop$# DECLARE
auto_workshop$#     current_stock INT;
auto_workshop$# BEGIN
auto_workshop$#     SELECT part.stock_balance INTO current_stock FROM part WHERE part.part_id = NEW.part_id;
auto_workshop$#     IF current_stock < NEW.quantity THEN
auto_workshop$#         RAISE EXCEPTION 'ОШИБКА: Недостаточно деталей на складе! В наличии: %', current_stock;
auto_workshop$#     END IF;
auto_workshop$#     UPDATE part SET stock_balance = part.stock_balance - NEW.quantity WHERE part.part_id = NEW.part_id;
auto_workshop$#     RETURN NEW;
auto_workshop$# END;
auto_workshop$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
auto_workshop=#
auto_workshop=# CREATE TRIGGER before_used_part_insert
auto_workshop=# BEFORE INSERT ON used_part
auto_workshop=# FOR EACH ROW
auto_workshop=# EXECUTE PROCEDURE trg_check_and_update_stock();
CREATE TRIGGER
auto_workshop=#
```

Рисунок 14 — Триггер



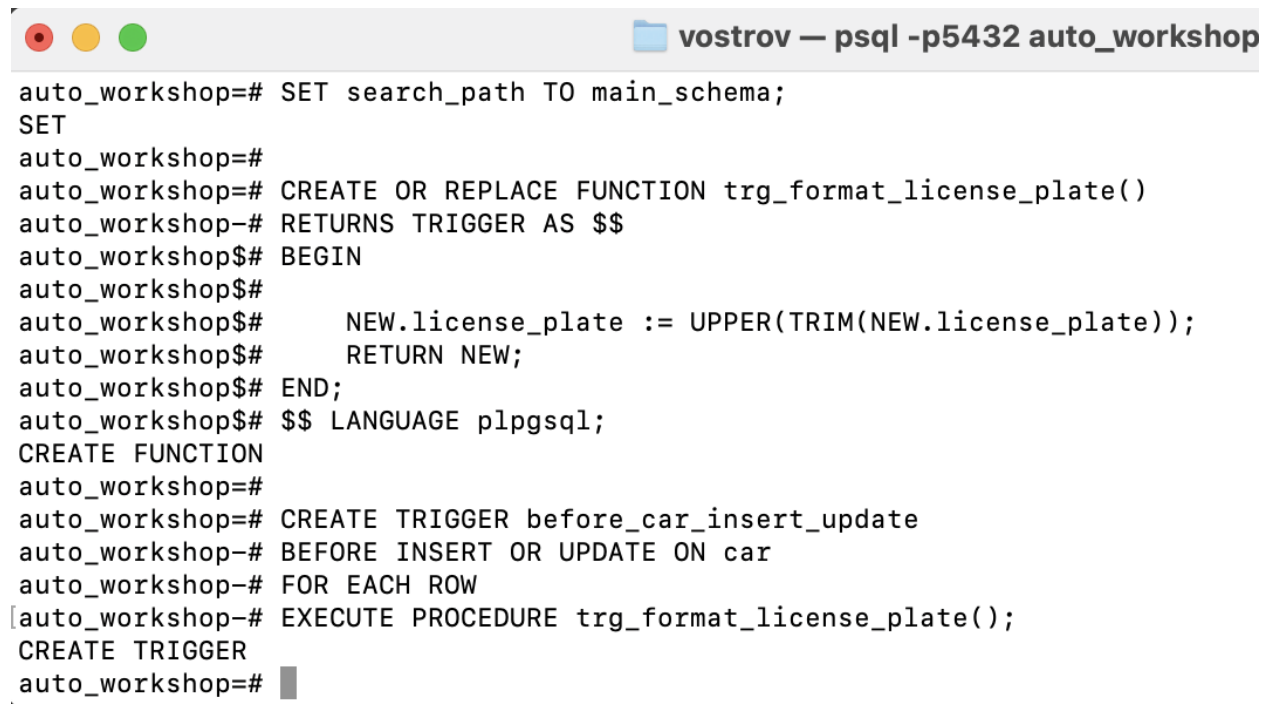
```
auto_workshop=# INSERT INTO used_part (contract_service_id, part_id, quantity, total_price)
auto_workshop=# VALUES ((SELECT min(contract_service_id) FROM contract_service), (SELECT min(part_id) FROM part), 67, 5000)
;
ERROR:  ОШИБКА: Недостаточно деталей на складе! В наличии: 10
CONTEXT:  PL/pgSQL function trg_check_and_update_stock() line 8 at RAISE
auto_workshop=#
```

Рисунок 15 — Результат

Триггер 5. Авто-форматирование госномера машины.

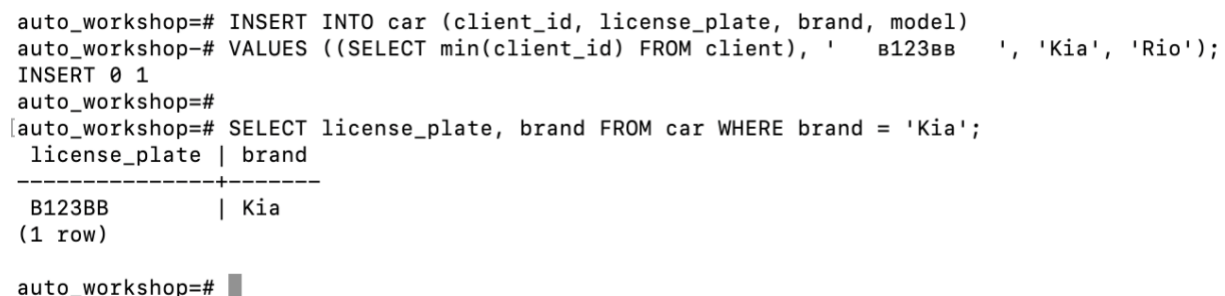
Цель: Привести все вводимые госномера к единому стандарту (верхний регистр, без пробелов).

Что я сделал: Триггер BEFORE INSERT OR UPDATE. Функция применяет встроенные методы UPPER() и TRIM() к объекту NEW.license_plate.



```
auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# CREATE OR REPLACE FUNCTION trg_format_license_plate()
auto_workshop=# RETURNS TRIGGER AS $$
auto_workshop$# BEGIN
auto_workshop$#
auto_workshop$#     NEW.license_plate := UPPER(TRIM(NEW.license_plate));
auto_workshop$#     RETURN NEW;
auto_workshop$# END;
auto_workshop$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
auto_workshop=#
auto_workshop=# CREATE TRIGGER before_car_insert_update
auto_workshop=# BEFORE INSERT OR UPDATE ON car
auto_workshop=# FOR EACH ROW
auto_workshop=# EXECUTE PROCEDURE trg_format_license_plate();
CREATE TRIGGER
auto_workshop=#
```

Рисунок 16 — Триггер



```
auto_workshop=# INSERT INTO car (client_id, license_plate, brand, model)
auto_workshop=# VALUES ((SELECT min(client_id) FROM client), '  B123BB  ', 'Kia', 'Rio');
INSERT 0 1
auto_workshop=#
auto_workshop=# SELECT license_plate, brand FROM car WHERE brand = 'Kia';
 license_plate | brand
-----+-----
 B123BB       | Kia
(1 row)

auto_workshop=#
```

Рисунок 17 — Результат

Триггер 6. Аудит изменения разряда механиков.

Цель: Сохранять историю кадровых перемещений.

Что я сделал: Триггер AFTER UPDATE. При изменении колонки rate_count функция записывает OLD и NEW значения в созданную таблицу аудита employee_audit.

```
vostrov — psql -p5432 auto_workshop — 123x31

auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# CREATE TABLE employee_audit (
auto_workshop(#      audit_id SERIAL PRIMARY KEY,
auto_workshop(#      employee_id INT,
auto_workshop(#      old_rank INT,
auto_workshop(#      new_rank INT,
auto_workshop(#      changed_at TIMESTAMP DEFAULT NOW()
auto_workshop(# );
CREATE TABLE
auto_workshop=#
auto_workshop=# CREATE OR REPLACE FUNCTION trg_audit_employee_rank()
auto_workshop=# RETURNS TRIGGER AS $$
auto_workshop$$ BEGIN
auto_workshop$$     IF OLD.rate_count IS DISTINCT FROM NEW.rate_count THEN
auto_workshop$$         INSERT INTO employee_audit (employee_id, old_rank, new_rank)
auto_workshop$$             VALUES (NEW.employee_id, OLD.rate_count, NEW.rate_count);
auto_workshop$$     END IF;
auto_workshop$$     RETURN NEW;
auto_workshop$$ END;
auto_workshop$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
auto_workshop=#
auto_workshop=# CREATE TRIGGER after_employee_update
auto_workshop=# AFTER UPDATE ON employee
auto_workshop=# FOR EACH ROW
auto_workshop=# EXECUTE PROCEDURE trg_audit_employee_rank();
CREATE TRIGGER
auto_workshop=#
```

Рисунок 18 — Триггер

```
auto_workshop=# UPDATE employee SET rate_count = 5 WHERE employee_id = (SELECT min(employee_id) FROM employee);
UPDATE 1
auto_workshop=#
auto_workshop=# SELECT * FROM employee_audit;
 audit_id | employee_id | old_rank | new_rank |          changed_at
-----+-----+-----+-----+-----
      1 |          1 |         |      5 | 2026-04-19 16:31:12.156206
(1 row)

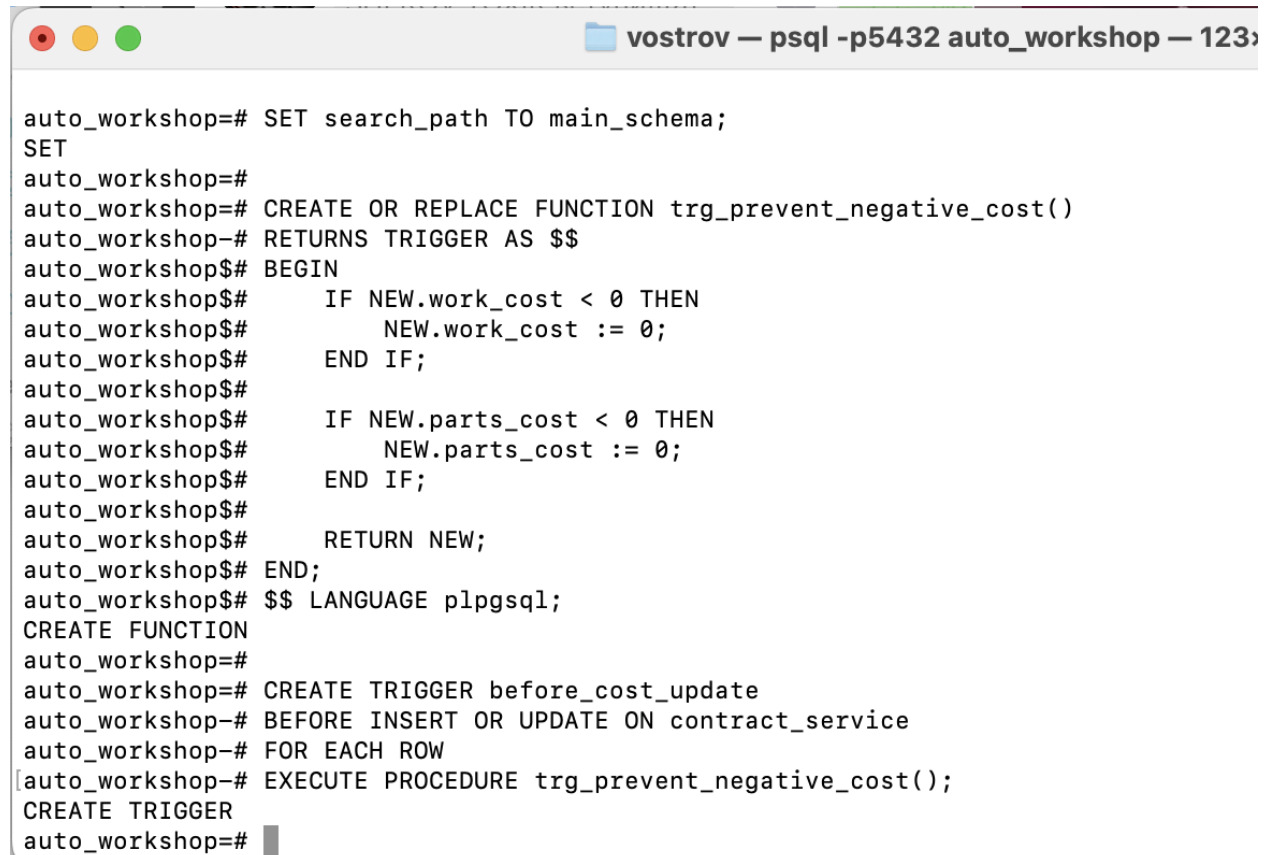
auto_workshop=#
```

Рисунок 19 — Результат

Триггер 7. Защита от отрицательной стоимости работ.

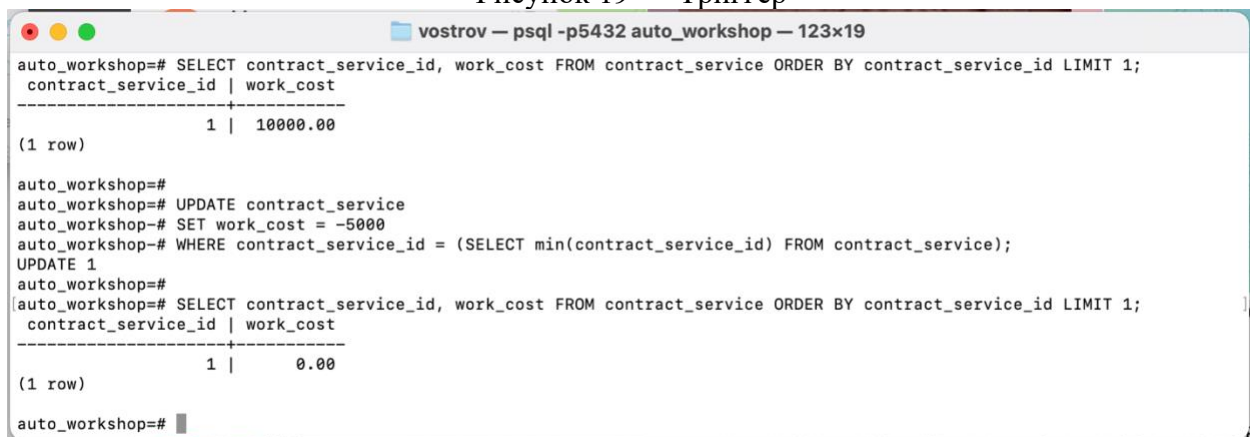
Цель: Защита от ошибок при заполнении стоимости ремонта.

Что я сделал: Триггер BEFORE INSERT OR UPDATE. Если NEW.work_cost < 0, функция не выдает ошибку, а заменяет отрицательное значение на 0, после чего разрешает запись.



```
vostrov — psql -p5432 auto_workshop — 123>
auto_workshop=# SET search_path TO main_schema;
SET
auto_workshop=#
auto_workshop=# CREATE OR REPLACE FUNCTION trg_prevent_negative_cost()
auto_workshop=# RETURNS TRIGGER AS $$
auto_workshop$$ BEGIN
auto_workshop$$     IF NEW.work_cost < 0 THEN
auto_workshop$$         NEW.work_cost := 0;
auto_workshop$$     END IF;
auto_workshop$$
auto_workshop$$     IF NEW.parts_cost < 0 THEN
auto_workshop$$         NEW.parts_cost := 0;
auto_workshop$$     END IF;
auto_workshop$$
auto_workshop$$     RETURN NEW;
auto_workshop$$ END;
auto_workshop$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
auto_workshop=#
auto_workshop=# CREATE TRIGGER before_cost_update
auto_workshop=# BEFORE INSERT OR UPDATE ON contract_service
auto_workshop=# FOR EACH ROW
[auto_workshop=# EXECUTE PROCEDURE trg_prevent_negative_cost();
CREATE TRIGGER
auto_workshop=#
```

Рисунок 19 — Триггер



```
vostrov — psql -p5432 auto_workshop — 123x19
auto_workshop=# SELECT contract_service_id, work_cost FROM contract_service ORDER BY contract_service_id LIMIT 1;
 contract_service_id | work_cost
-----
1 | 10000.00
(1 row)

auto_workshop=#
auto_workshop=# UPDATE contract_service
auto_workshop=# SET work_cost = -5000
auto_workshop=# WHERE contract_service_id = (SELECT min(contract_service_id) FROM contract_service);
UPDATE 1
auto_workshop=#
auto_workshop=# SELECT contract_service_id, work_cost FROM contract_service ORDER BY contract_service_id LIMIT 1;
 contract_service_id | work_cost
-----
1 | 0.00
(1 row)

auto_workshop=#
```

Рисунок 20 — Результат

ВЫВОДЫ

В ходе выполнения лабораторной работы в консольном клиенте `psql` были успешно освоены механизмы серверного программирования в СУБД PostgreSQL.

На практике изучено создание Хранимых процедур для выполнения сложных многотабличных операций обновления и Хранимых функций для генерации табличных отчетов. Разработаны 7 оригинальных триггеров уровней BEFORE и AFTER, которые позволили автоматизировать каскадные пересчеты цен, внедрить надежные ограничения целостности и организовать прозрачную систему аудита изменений в БД.